

فاز ۱۴: Servlet و مدیریت Session – کامل و عمیق



این بخش برای درک مفاهیم پایه‌ای وب در جاوا ضروریه. مفاهیمی مثل Stateless، Filter، Session، ServletContext و Token از سوالات پرتکرار مصاحبه‌های بک‌اند هستند.

۱۴.۱ تفاوت HTTP و FTP (Stateless vs Stateful)

! سوال مصاحبه‌ای ۱: HTTP Stateless هست یا Stateful؟ FTP چگونه؟

HTTP Stateless

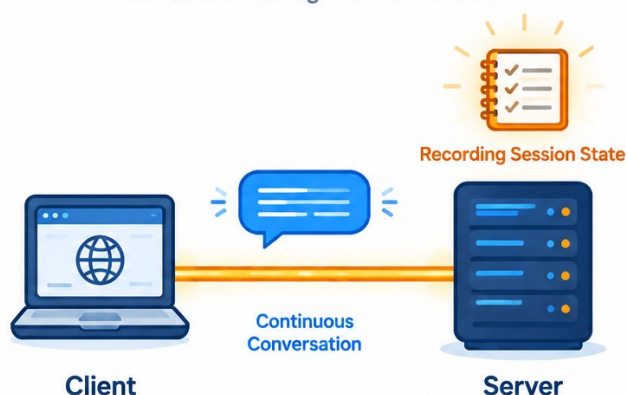
Each request is independent.
No memory between requests.



3 separate, independent requests

FTP Stateful

Maintains a continuous connection.
Shares state throughout the session.



One connection, shared state

جواب:

ویژگی	HTTP	FTP
وضعیت (State)	Stateless (بدون حالت)	Stateful (با حالت)
معنی	هر درخواست مستقل از درخواست قبلیه. سرور هیچ چیزی از درخواست قبلی به خاطر نمی‌سپاره.	سرور وضعیت اتصال رو نگه می‌داره. مثلاً می‌دونه الان توی چه پوشه‌ای هستی.
اتصال	معمولاً Connectionless (هر درخواست یه اتصال جدید)	Connection-Oriented (یه اتصال باز می‌مونه)
پورت پیش‌فرض	80 (HTTP)، 443 (HTTPS)	20 و 21
کاربرد	وب، REST API	انتقال فایل

چرا HTTP Stateless طراحی شده؟

- سادگی و مقیاس‌پذیری (Scalability)
- سرور مجبور نیست برای هر کاربر حافظه اختصاص بده
- هر درخواست می‌تونه به هر سروری توی Cluster بره

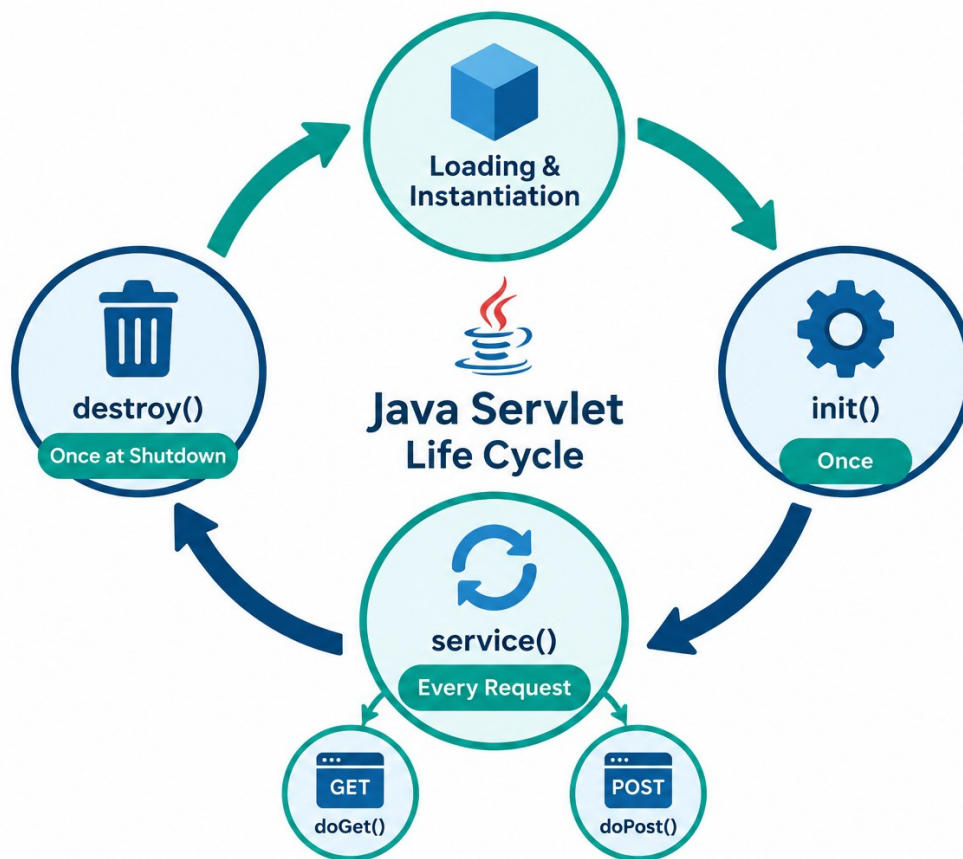
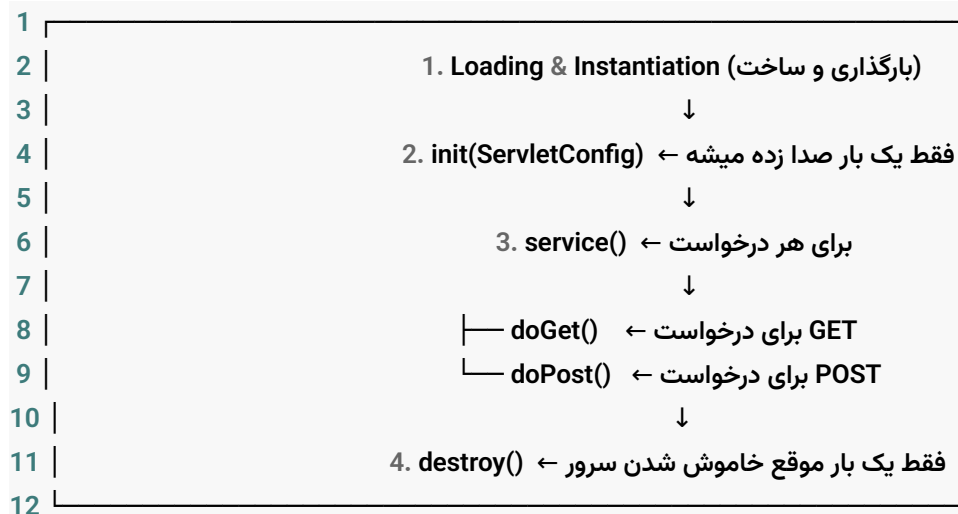
مشکل Stateless بودن HTTP چیه؟

- سرور نمی‌دونه کاربر قبلاً Login کرده یا نه!
- راه‌حل: Session یا Token (JWT) برای نگه‌داشتن وضعیت.

۱۴.۲ Servlet چیست؟

Servlet یه کلاس جاواییه که روی سرور اجرا میشه و درخواست‌های HTTP رو پردازش می‌کنه. Servlet Container (مثل Tomcat) مسئول مدیریت چرخه حیات Servlet‌هاست.

چرخه حیات Servlet



```

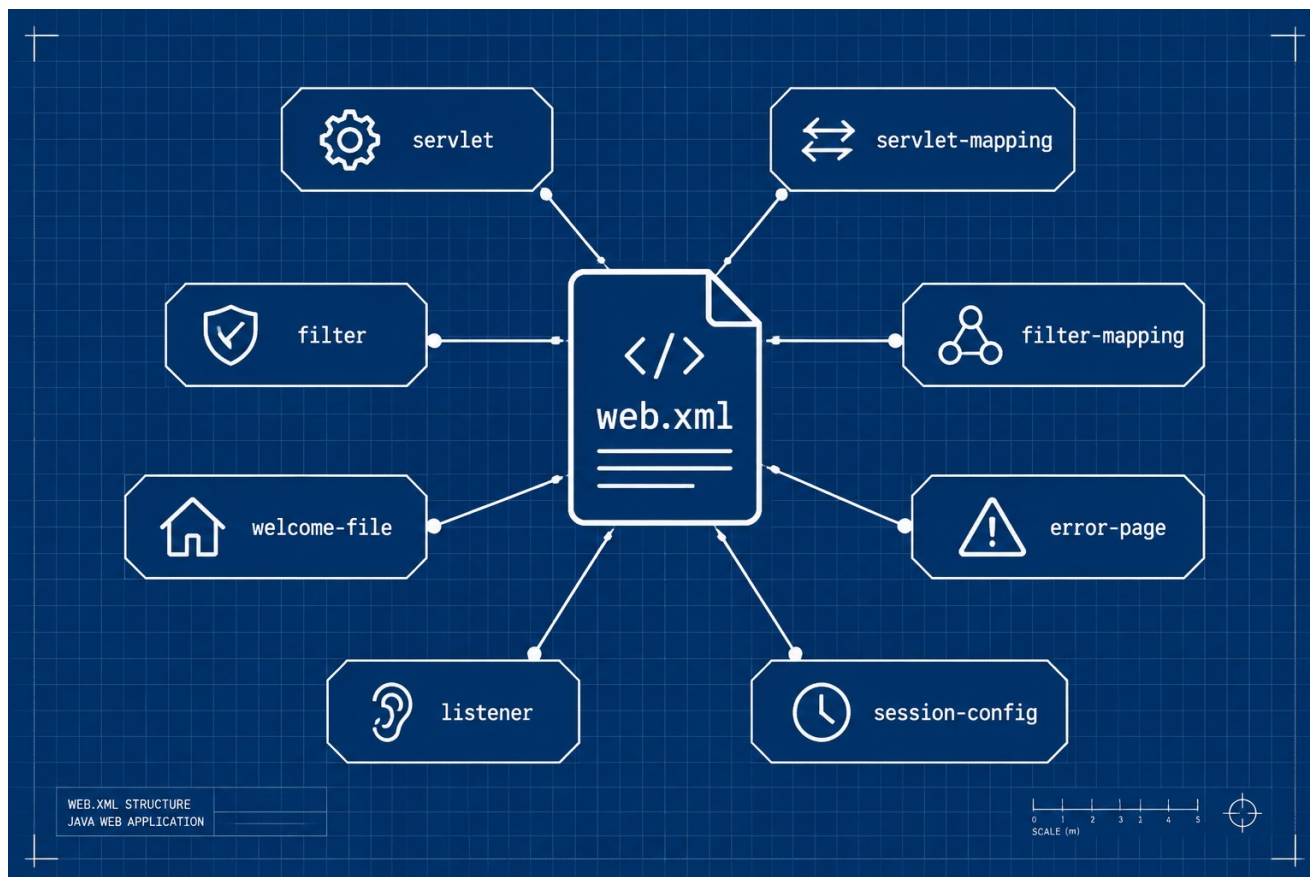
1 public class LoginServlet extends HttpServlet {
2
3     @Override
4     public void init() throws ServletException {
5         System.out.println("1. Servlet داره initialize همیشه");
6     }
7
8     @Override
9     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
10         throws ServletException, IOException {
11         resp.getWriter().println("دریافت شد GET درخواست");
12     }
13
14     @Override
15     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
16         throws ServletException, IOException {
17         String username = req.getParameter("username");
18         String password = req.getParameter("password");
19         // پردازش لاگین
20     }
21
22     @Override
23     public void destroy() {
24         System.out.println("4. Servlet داره destroy همیشه");
25     }
26 }

```

۱۴.۳ web.xml – قلب تنظیمات Servlet

فایل تنظیمات Deployment Descriptor هست. توی پروژه‌های قدیمی و مصاحبه‌ها خیلی مهمه.

ساختار کامل web.xml



```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
5     http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
6     version="4.0">
7
8     <!-- Servlet تعریف -->
9     <servlet>
10         <servlet-name>login</servlet-name>
11         <servlet-class>ir.nas.LoginServlet</servlet-class>
12
13     <!-- init-param: پارامترهای اولیه برای Servlet -->
14     <init-param>

```

```

15     <param-name>dbUrl</param-name>
16     <param-value>jdbc:mysql://localhost:3306/mydb</param-value>
17 </init-param>
18 <init-param>
19     <param-name>maxRetries</param-name>
20     <param-value>3</param-value>
21 </init-param>
22
23 <!-- load-on-startup: موقع استارت سرور ساخته میشه -->
24 <load-on-startup>1</load-on-startup>
25 </servlet>
26
27 <!-- ۲. Mapping Servlet به URL -->
28 <servlet-mapping>
29     <servlet-name>login</servlet-name>
30     <url-pattern>/login</url-pattern>
31 </servlet-mapping>
32
33 <!-- ۳. تعریف Filter -->
34 <filter>
35     <filter-name>filter-login</filter-name>
36     <filter-class>ir.nas.LoginFilter</filter-class>
37
38 <!-- init-param برای Filter -->
39 <init-param>
40     <param-name>excludedPaths</param-name>
41     <param-value>/public,/css,/js</param-value>
42 </init-param>
43 </filter>
44
45 <!-- ۴. Mapping Filter به URL -->
46 <filter-mapping>
47     <filter-name>filter-login</filter-name>
48     <url-pattern>/login</url-pattern>
49     <!-- کنی map خاص هم Servlet می‌تونن به -->
50     <!-- <servlet-name>login</servlet-name> -->
51 </filter-mapping>

```

```

52
53 <!-- ۵. Listener -->
54 <listener>
55   <listener-class>ir.nas.MyAppContextListener</listener-class>
56 </listener>
57
58 <!-- ۶. تنظیمات Session -->
59 <session-config>
60   <session-timeout>30</session-timeout> <!-- دقیقه -->
61   <cookie-config>
62     <http-only>true</http-only>
63     <secure>false</secure>
64   </cookie-config>
65 </session-config>
66
67 <!-- ۷. صفحه خوش آمد -->
68 <welcome-file-list>
69   <welcome-file>index.html</welcome-file>
70   <welcome-file>index.jsp</welcome-file>
71 </welcome-file-list>
72
73 <!-- ۸. صفحه خطا -->
74 <error-page>
75   <error-code>404</error-code>
76   <location>/404.html</location>
77 </error-page>
78 <error-page>
79   <error-code>500</error-code>
80   <location>/500.html</location>
81 </error-page>
82
83 </web-app>

```

دسترسی به init-param توی Servlet

```

1 public class LoginServlet extends HttpServlet {
2   private String dbUrl;

```

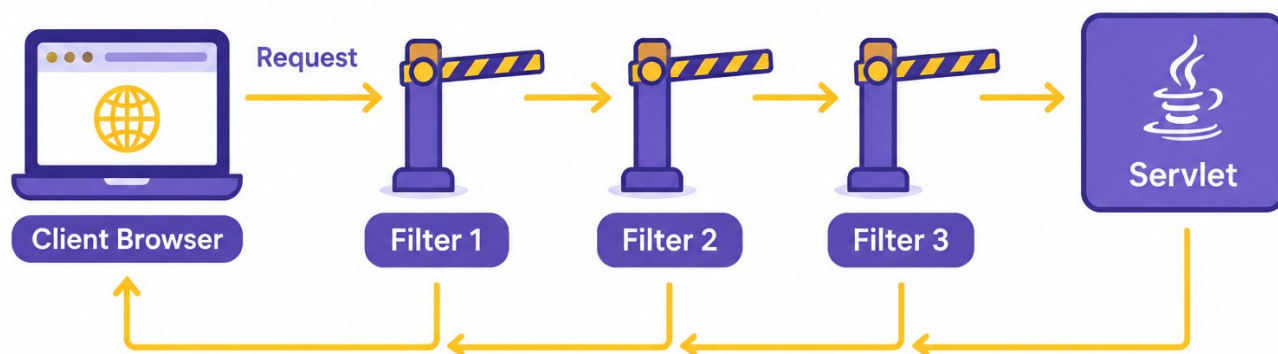
```

3 private int maxRetries;
4
5 @Override
6 public void init(ServletConfig config) throws ServletException {
7     // روش ۱: از طریق ServletConfig
8     this.dbUrl = config.getInitParameter("dbUrl");
9     this.maxRetries = Integer.parseInt(config.getInitParameter("maxRetries"));
10
11     // روش ۲: از طریق getServletConfig()
12     String name = getServletConfig().getServletName();
13 }
14 }

```

۱۴.۴ Filter – نگهبان درخواست‌ها

Filter مثل یه دروازه‌بان عمل می‌کنه: قبل از رسیدن درخواست به Servlet اجرا میشه و می‌تونه درخواست رو رد کنه، تغییر بده یا لاگ بگیره.



چرخه حیات Filter

1 Request → Filter1 → Filter2 → Servlet → Filter2 → Filter1 → Response

نمونه LoginFilter

```

1 public class LoginFilter implements Filter {
2
3     private List<String> excludedPaths;
4
5     @Override
6     public void init(FilterConfig config) throws ServletException {
7         // خواندن init-param
8         String paths = config.getInitParameter("excludedPaths");
9         this.excludedPaths = Arrays.asList(paths.split(","));
10        System.out.println("Filter initialized");
11    }
12
13    @Override
14    public void doFilter(ServletRequest request, ServletResponse response,
15                        FilterChain chain) throws IOException, ServletException {
16
17        HttpServletRequest req = (HttpServletRequest) request;
18        HttpServletResponse resp = (HttpServletResponse) response;
19
20        String path = req.getRequestURI();
21
22        // بود، رد کن excludedPaths اگر مسیر توی
23        if (excludedPaths.stream().anyMatch(path::startsWith)) {
24            chain.doFilter(request, response);
25            return;
26        }
27
28        // چک کردن Session
29        HttpSession session = req.getSession(false);
30        boolean isLoggedIn = (session != null && session.getAttribute("user") != null);

```

```

31
32     if (isLoggedIn) {
33         // کاربر لاگین کرده، برو به Servlet
34         chain.doFilter(request, response);
35     } else {
36         // کاربر لاگین نکرده، بفرست به صفحه لاگین
37         resp.sendRedirect(req.getContextPath() + "/login.jsp");
38     }
39 }
40
41 @Override
42 public void destroy() {
43     System.out.println("Filter destroyed");
44 }
45 }

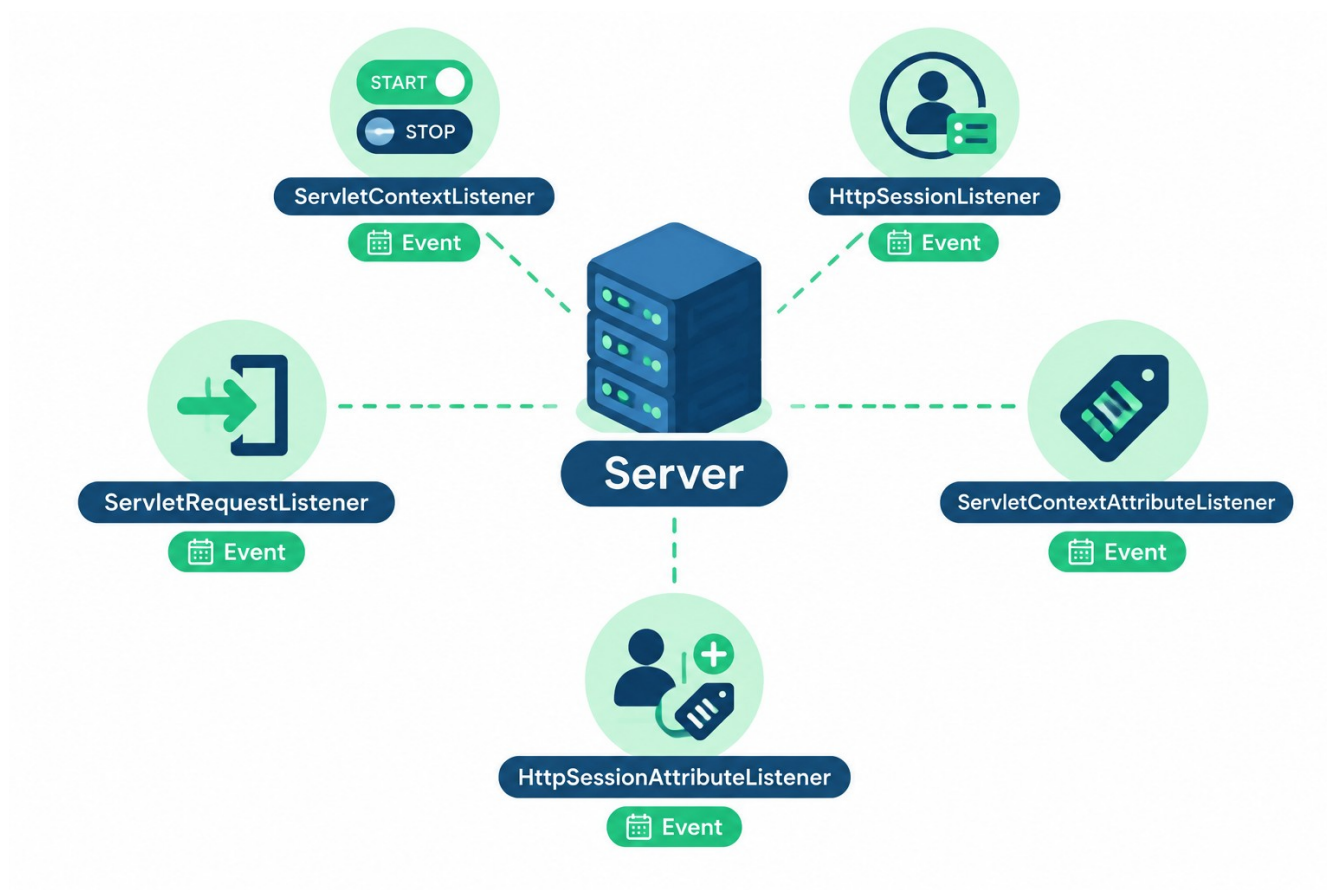
```

ترتیب اجرای Filterها

ترتیب توی web.xml مهمه! Filter ها به ترتیبی که تعریف میشن اجرا میشن.

۱۴.۵ Listener – شنونده رویدادها

Listener کلاسیه که به رویدادهای چرخه حیات وب اپ گوش میده.



انواع Listener

Listener	رویداد
ServletContextListener	استارت/خاموشی وب اپ
HttpSessionListener	ساخت/نابودی Session
ServletRequestListener	شروع/پایان درخواست
ServletContextAttributeListener	تغییر attribute در Context
HttpSessionAttributeListener	تغییر attribute در Session

نمونه ServletContextListener

```

1 java
2 public class MyAppContextListener implements ServletContextListener {
3
4     @Override
5     public void contextInitialized(ServletContextEvent sce) {
6         System.out.println("وب اپ داره استارت میشه");
7
8         // مثلاً راه اندازی Connection Pool
9         ServletContext ctx = sce.getServletContext();
10        ctx.setAttribute("appName", "MyApp");
11    }
12
13    @Override
14    public void contextDestroyed(ServletContextEvent sce) {
15        System.out.println("وب اپ داره خاموش میشه");
16        // بستن منابع
17    }
18 }

```

نمونه HttpSessionListener

```

1 java
2 public class MySessionListener implements HttpSessionListener {
3
4     private static int activeSessions = 0;
5
6     @Override
7     public void sessionCreated(HttpSessionEvent se) {
8         activeSessions++;
9         System.out.println("جدید ساخته شد. تعداد Session: " + activeSessions);
10    }
11
12    @Override
13    public void sessionDestroyed(HttpSessionEvent se) {

```

```

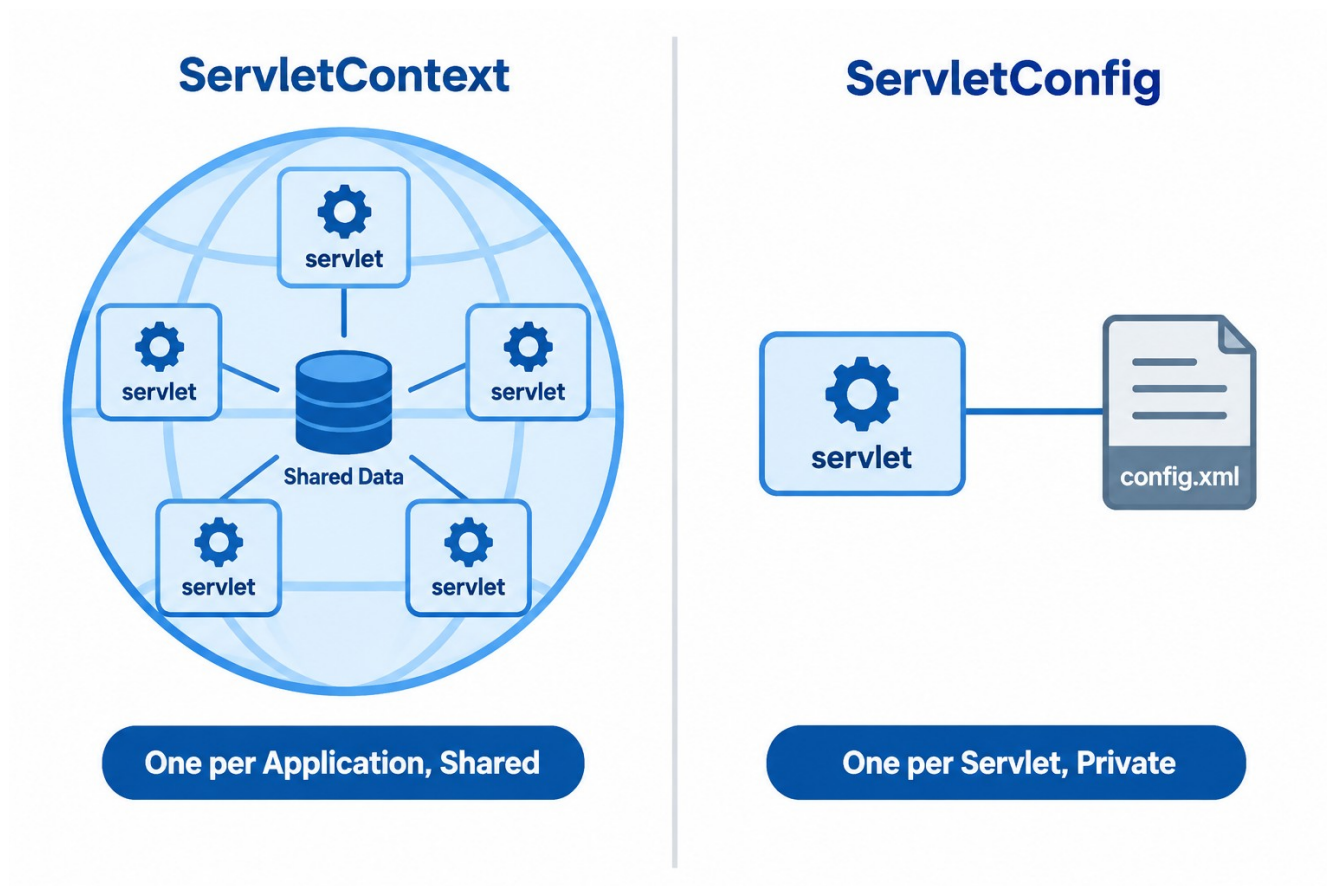
14  activeSessions--;
15  System.out.println("Session تعداد شد. نابود شد. " + activeSessions);
16  }
17  }

```

ServletConfig و ServletContext ۱۴.۶

! سوال مصاحبه‌ای ۲: فرق ServletConfig و ServletContext چیه؟

جواب:



14-5-Servlet

ویژگی	ServletConfig	ServletContext
دامنه	مخصوص به Servlet خاص	کل وب اپ
تعداد	یکی برای هر Servlet	یکی برای کل اپ
دسترسی	getServletConfig()	getServletContext()
init-param	مخصوص همون Servlet	برای کل اپ
کاربرد	تنظیمات خاص به Servlet	اشتراک داده بین همه Servletها

1 java

```

2 // ServletConfig - مخصوص به Servlet
3 public class MyServlet extends HttpServlet {
4     @Override
5     public void init(ServletConfig config) {
6         String value = config.getInitParameter("myParam"); // فقط برای این Servlet
7     }
8 }
9
10 // ServletContext - برای کل اپ
11 public class AnotherServlet extends HttpServlet {
12     @Override
13     protected void doGet(HttpServletRequest req, HttpServletResponse resp) {
14         ServletContext ctx = getServletContext();
15         ctx.setAttribute("sharedData", "value"); // همه Servlet ها می بینن
16
17         String appName = ctx.getInitParameter("appName"); // پارامتر کل اپ
18     }
19 }

```

تنظیم context-param توی web.xml

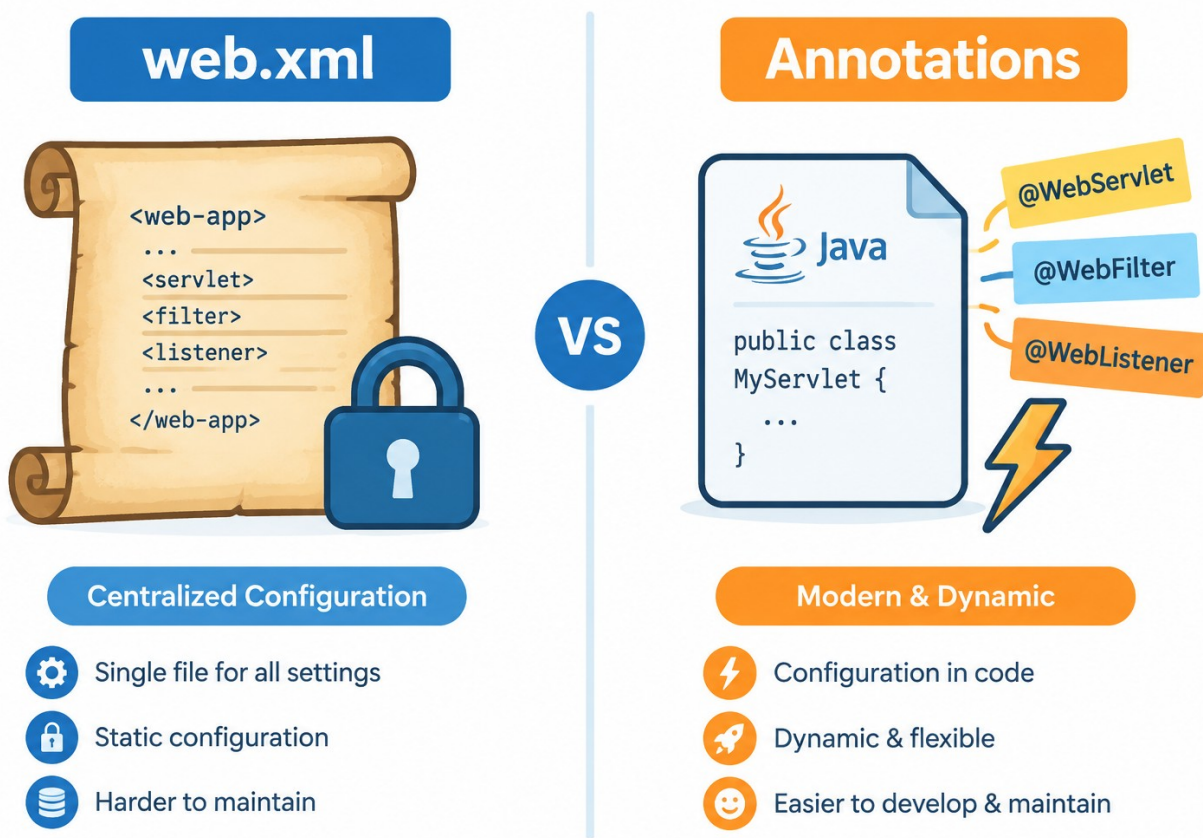
```

1 xml
2 <context-param>
3   <param-name>appName</param-name>
4   <param-value>MyApplication</param-value>
5 </context-param>

```

۱۴.۷ Annotation ها – جایگزین مدرن web.xml

از Servlet 3.0 به بعد، می‌تونن به جای web.xml از Annotation استفاده کنن.



WebServlet@

```

1 java
2 @WebServlet(
3     name = "login",
4     urlPatterns = {"/login", "/signin"},
5     initParams = {
6         @WebInitParam(name = "dbUrl", value = "jdbc:mysql://localhost:3306/mydb"),
7         @WebInitParam(name = "maxRetries", value = "3")
8     },
9     loadOnStartup = 1
10 )
11 public class LoginServlet extends HttpServlet {
12     // ...
13 }

```

WebFilter@

```

1 java
2 @WebFilter(
3     filterName = "filter-login",
4     urlPatterns = {"/login", "/admin/*"},
5     initParams = {
6         @WebInitParam(name = "excludedPaths", value = "/public,/css,/js")
7     }
8 )
9 public class LoginFilter implements Filter {
10     // ...
11 }

```

WebListener@

```

1 java
2 @WebListener
3 public class MySessionListener implements HttpSessionListener { // ... }

```

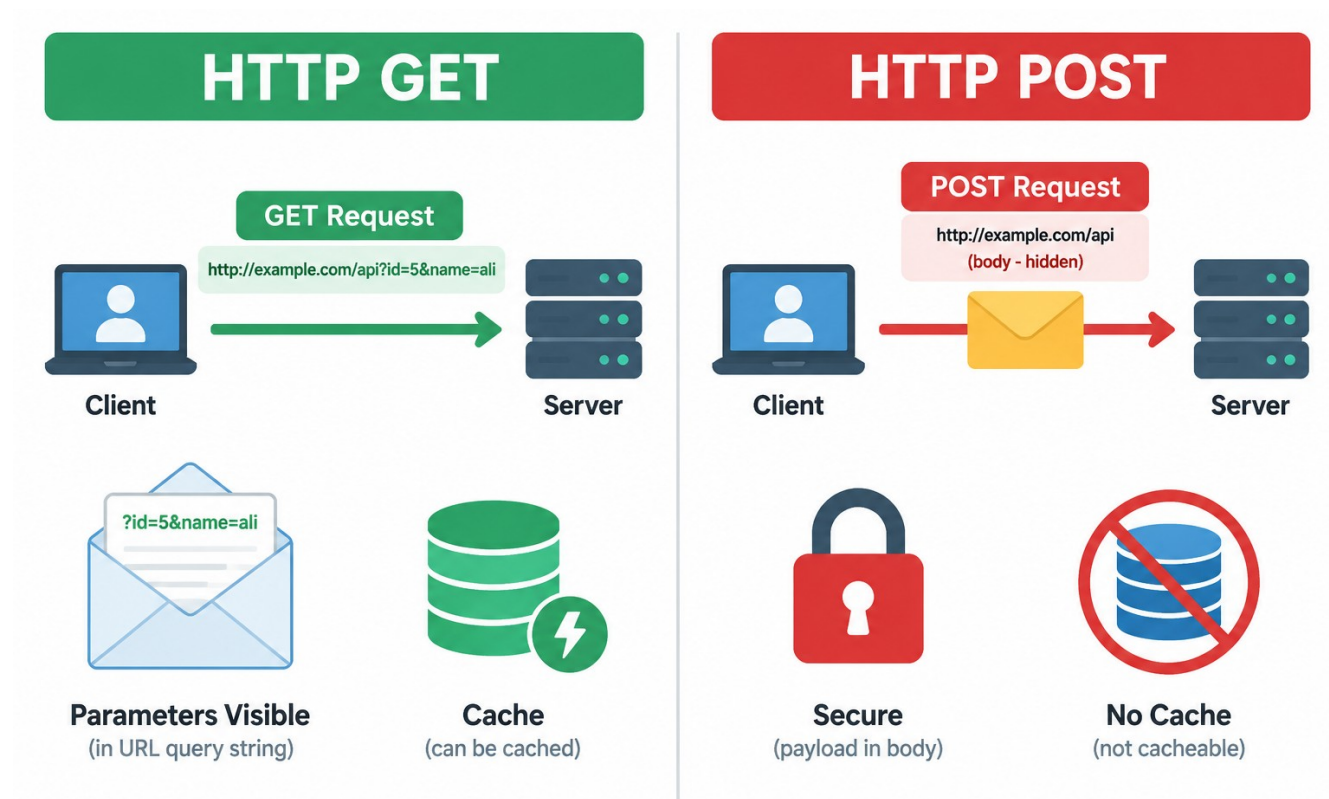
! سوال مصاحبه‌ای ۳: web.xml یا Annotation - کدام بهتره؟

جواب:

ویژگی	web.xml	Annotation
خوانایی	همه تنظیمات یه جا	پراکنده توی کد
تغییر بدون Recompile	✓ همیشه	✗ همیشه
محیط‌های مختلف	راحت (چند web.xml)	سخت
مدرن بودن	قدیمی	✓ مدرن
Spring Boot	استفاده نمیشه	✓ استفاده همیشه

توصیه: برای پروژه‌های جدید Annotation، برای تنظیمات محیطی web.xml.

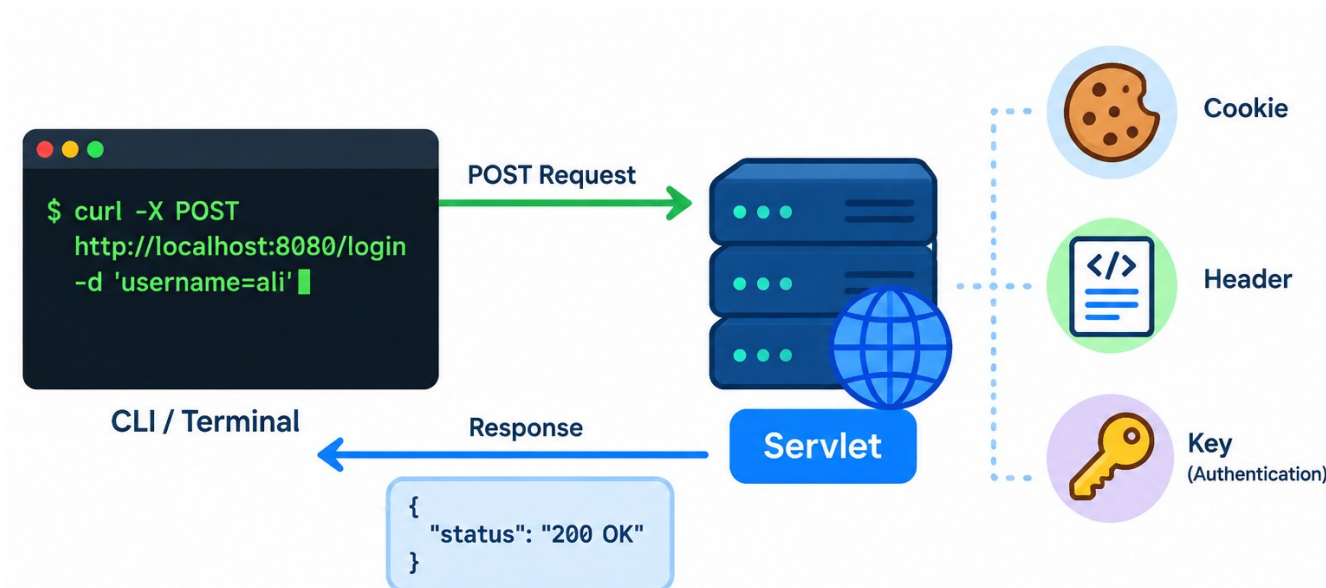
۱۴.۸ درخواست GET و POST (با curl)



تفاوت GET و POST

ویژگی	GET	POST
داده	توی URL (Query String)	توی Body
امنیت	❌ داده توی URL دیده میشه	✅ نسبی بهتر
طول	محدود (حدود 2000 کاراکتر)	نامحدود
Cache	✅ Cache میشه	❌ Cache نمیشه
Idempotent	✅ بله	❌ خیر
کاربرد	دریافت داده	ارسال داده، تغییر وضعیت

نمونه درخواست با curl



```
1 bash
2 # GET ساده
3 curl http://localhost:8080/api/users
4
5 # GET با Query Parameter
6 curl "http://localhost:8080/api/users?id=5&name=ali"
7
8 # GET با Header
9 curl -H "Authorization: Bearer token123" http://localhost:8080/api/users
10
11 # POST با فرم
12 curl -X POST http://localhost:8080/login \
13     -d "username=ali&password=1234"
14
15 # POST با JSON
16 curl -X POST http://localhost:8080/api/users \
17     -H "Content-Type: application/json" \
18     -d '{"name":"Ali","age":30}'
19
20 # POST با Basic Auth
21 curl -X POST http://localhost:8080/login \
22     -u ali:1234
23
24 # ذخیره کوکی
25 curl -c cookies.txt http://localhost:8080/login
26
27 # ارسال کوکی
28 curl -b cookies.txt http://localhost:8080/dashboard
29
30 # های پاسخ Header نمایش
31 curl -i http://localhost:8080/api/users
```

خوندن داده توی Servlet

```

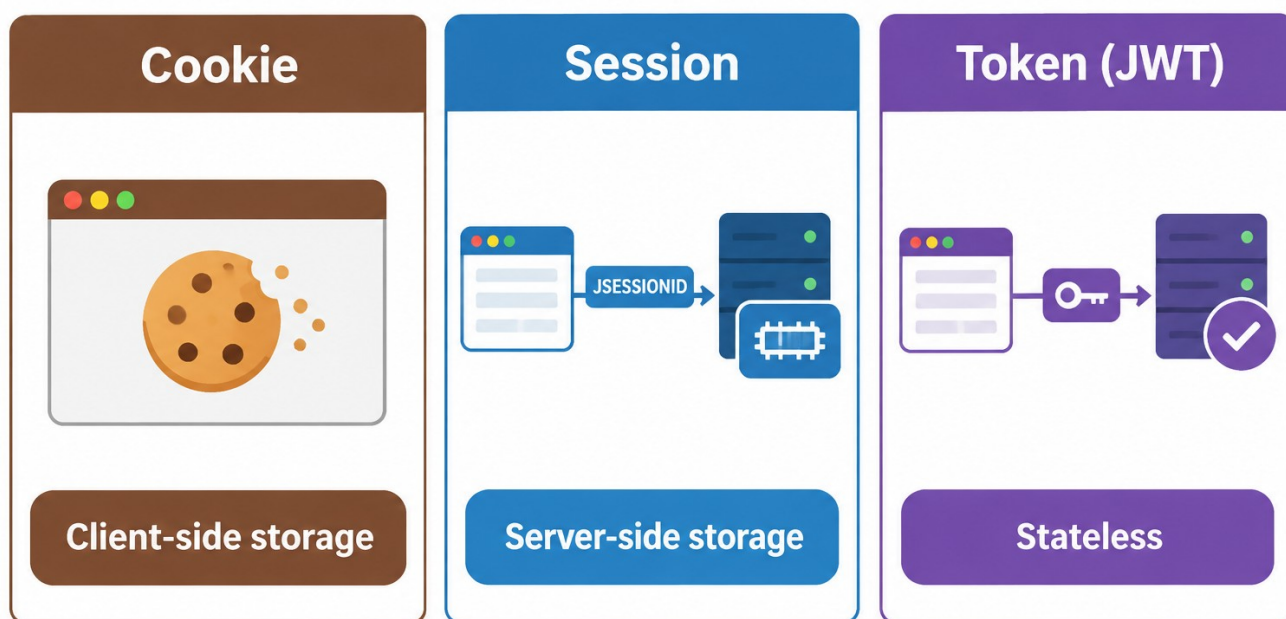
1 java
2 public class LoginServlet extends HttpServlet {
3
4     @Override
5     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
6         throws IOException {
7         // خوندن Query Parameter
8         String id = req.getParameter("id");
9         String name = req.getParameter("name");
10
11         resp.setContentType("text/html");
12         resp.getWriter().println("ID: " + id + ", Name: " + name);
13     }
14
15     @Override
16     protected void doPost(HttpServletRequest req, HttpServletResponse resp)
17         throws IOException {
18         // خوندن فرم
19         String username = req.getParameter("username");
20         String password = req.getParameter("password");
21
22         // خوندن JSON از Body
23         BufferedReader reader = req.getReader();
24         StringBuilder json = new StringBuilder();
25         String line;
26         while ((line = reader.readLine()) != null) {
27             json.append(line);
28         }
29
30         // خوندن Header
31         String authHeader = req.getHeader("Authorization");
32
33         resp.getWriter().println("Login attempt: " + username);
34     }
35 }

```

Cookie و Token، Session ۱۴.۹

! سوال مصاحبه‌ای ۴: تفاوت Cookie، Session و Token چیست؟

جواب:



ویژگی	Cookie	Session	Token (JWT)
محل ذخیره	مرورگر کاربر	سرور	مرورگر (localStorage)
Stateless	-	Stateful ❌	Stateless ✅
امنیت	کم	متوسط	بالا (اگر درست پیاده بشه)
مقیاس پذیری	-	سخت (Sticky Session) ❌	راحت ✅
انقضا	قابل تنظیم	قابل تنظیم	داخل Token
کاربرد	ذخیره تنظیمات	لاگین سنتی	REST API مدرن

Cookie

کوکی یه فایل کوچیکه که سرور توی مرورگر کاربر ذخیره می‌کنه.

```

1 java
2 // ساخت Cookie
3 Cookie cookie = new Cookie("username", "ali");
4 cookie.setMaxAge(60 * 60 * 24); // ۱ روز
5 cookie.setHttpOnly(true); // جلوگیری از دسترسی JS
6 cookie.setSecure(true); // فقط HTTPS
7 cookie.setPath("/");
8 resp.addCookie(cookie);
9
10 // خواندن Cookie
11 Cookie[] cookies = req.getCookies();
12 if (cookies != null) {
13     for (Cookie c : cookies) {
14         if (c.getName().equals("username")) {
15             String username = c.getValue();
16         }
17     }
18 }
19
20 // حذف Cookie
21 Cookie delete = new Cookie("username", "");
22 delete.setMaxAge(0);
23 resp.addCookie(delete);

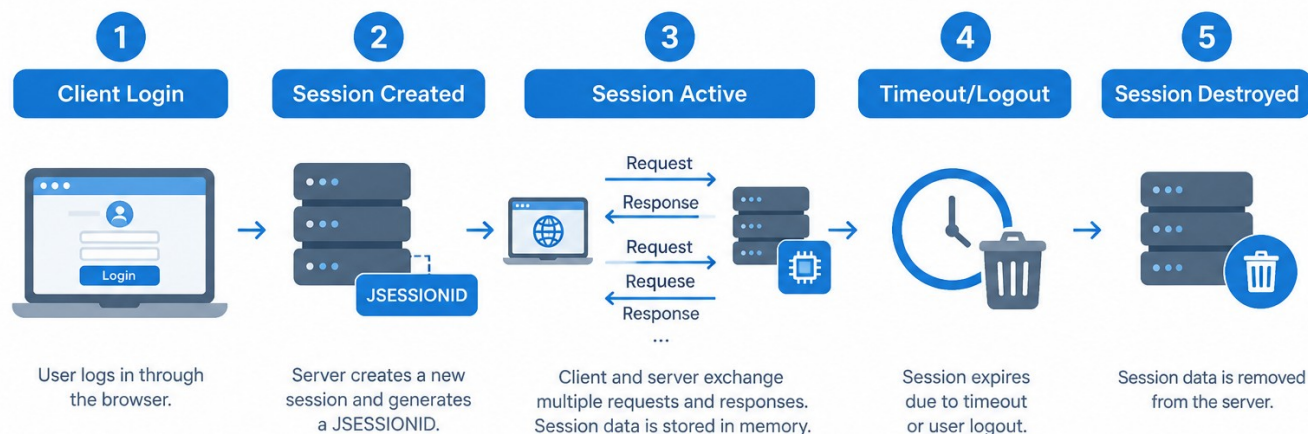
```

Session

Session یه شیء سمت سرور که اطلاعات کاربر رو نگه می‌داره. مرورگر فقط یه شناسه (JSESSIONID) رو توی Cookie نگه می‌داره.

Session Lifecycle

in a Java Web Application



1 java

2 // ساخت/گرفتن Session

3 HttpSession session = req.getSession(true); // اگه نبود، بساز

4 session.setAttribute("user", userObject);

5 session.setMaxInactiveInterval(30 * 60); // دقیقه ۳۰

6

7 // خواندن Session

8 HttpSession session = req.getSession(false); // بده null، اگه نبود

9 if (session != null) {

10 User user = (User) session.getAttribute("user");

11 }

12

13 // حذف Session

14 session.invalidate();

15

16 // Timeout

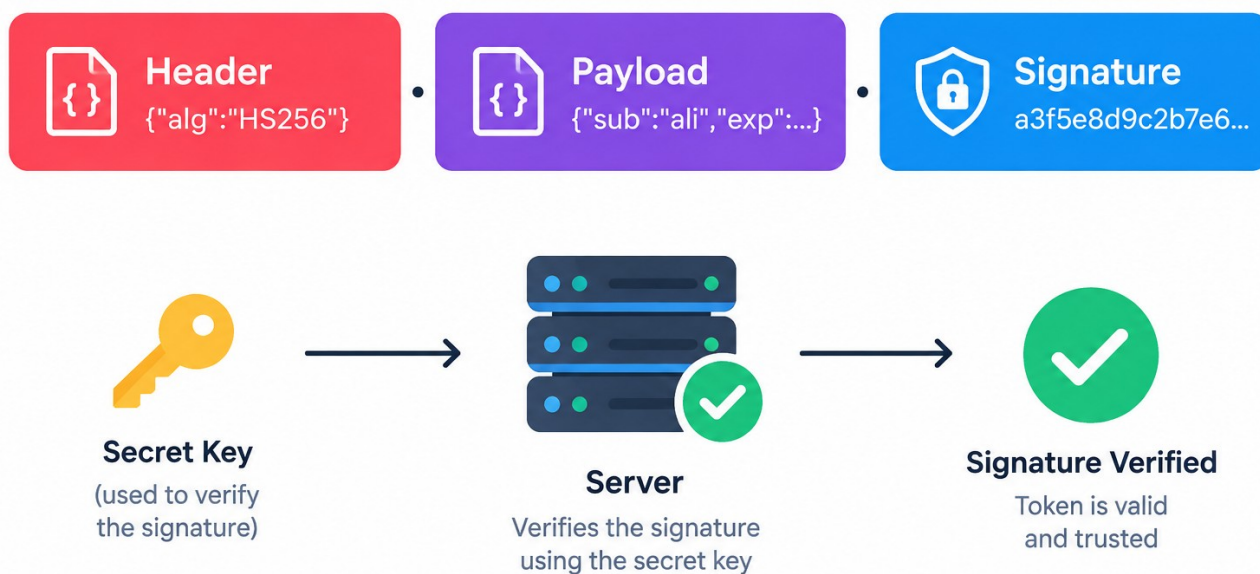
17 session.setMaxInactiveInterval(60 * 30); // دقیقه ۳۰

Token (JWT)

JWT به رشته‌ست که سه بخش دارد: Header.Payload.Signature

JWT (JSON Web Token)

A JWT is made up of three parts: Header, Payload and Signature.



```
1 java
```

```
2 // ساخت JWT (با کتابخانه jjwt)
```

```
3 String token = Jwts.builder()
```

```
4   .setSubject("ali")
```

```
5   .setIssuedAt(new Date())
```

```
6   .setExpiration(new Date(System.currentTimeMillis() + 3600000))
```

```
7   .signWith(SignatureAlgorithm.HS256, "secretKey")
```

```
8   .compact();
```

```
9
```

```
10 // خواندن JWT
```

```
11 String authHeader = req.getHeader("Authorization");
```

```
12 if (authHeader != null && authHeader.startsWith("Bearer ")) {
```

```
13   String token = authHeader.substring(7);
```

```

14 Claims claims = Jwts.parser()
15     .setSigningKey("secretKey")
16     .parseClaimsJws(token)
17     .getBody();
18 String username = claims.getSubject();
19 }
20
21 // به کلاینت Token ارسال
22 resp.setHeader("Authorization", "Bearer " + token);

```

! سوال مصاحبه‌ای ۵: Session-based یا Token-based – کدام بهتره؟

جواب:

:Session-based

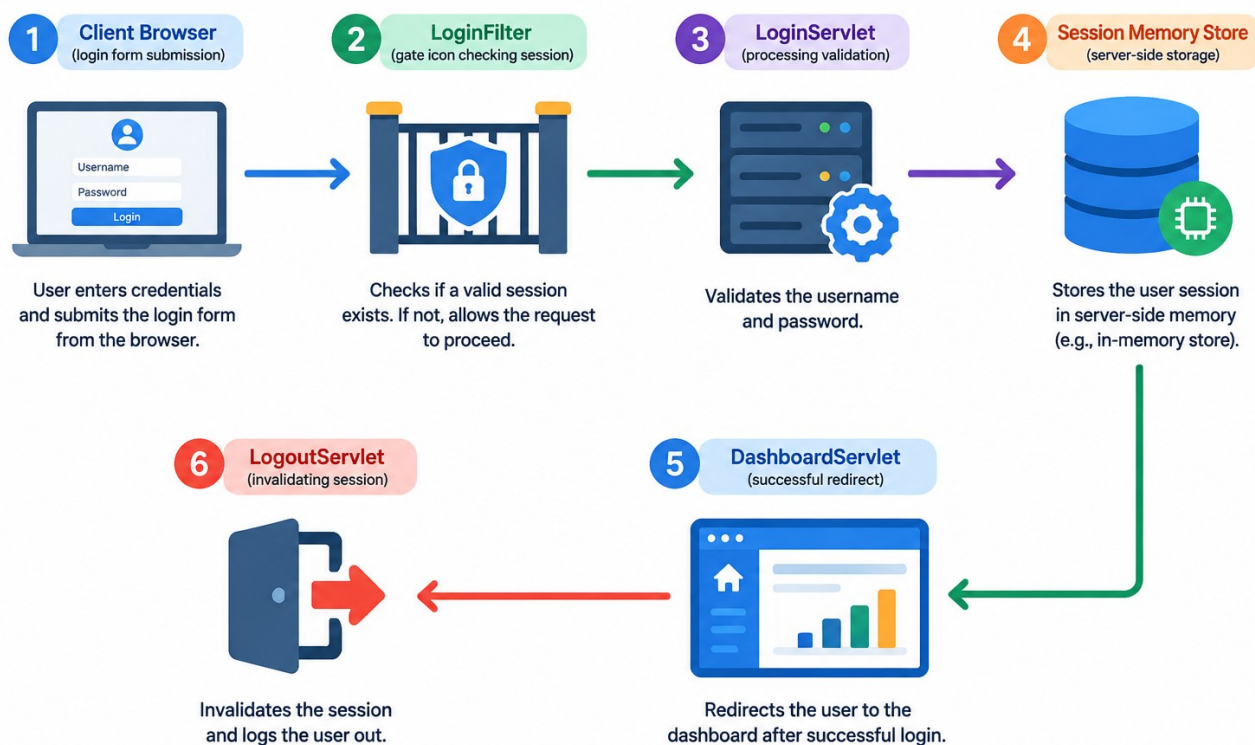
- ساده‌تر ✓
- می‌تونی هر لحظه Logout کنی (invalidate) ✓
- Stateful – برای Cluster نیاز به Sticky Session یا Session Replication ✗
- برای Mobile App ها مناسب نیست ✗

:Token-based (JWT)

- Stateless – مقیاس‌پذیری عالی ✓
- برای REST API و Mobile عالی ✓
- Cross-domain راحت ✓
- نمی‌تونی قبل از Token، Expire، رو باطل کنی (مگر با Blacklist) ✗
- اگر Secret Leak بشه، همه چیز از دست میره ✗

توصیه: برای REST API مدرن، JWT. برای وب اپ سنتی، Session.

۱۴.۱۰ نمونه کامل: Login با Session + Filter + Servlet



LoginServlet.java

```

1 java
2 @WebServlet("/login")
3 public class LoginServlet extends HttpServlet {
4
5     @Override
6     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
7         throws ServletException, IOException {
8         // نمایش فرم لاگین
  
```

```

9     req.getRequestDispatcher("/login.jsp").forward(req, resp);
10 }
11
12 @Override
13 protected void doPost(HttpServletRequest req, HttpServletResponse resp)
14     throws ServletException, IOException {
15
16     String username = req.getParameter("username");
17     String password = req.getParameter("password");
18
19     // اعتبارسنجی (ساده)
20     if ("ali".equals(username) && "1234".equals(password)) {
21         // لاگین موفق
22         HttpSession session = req.getSession(true);
23         session.setAttribute("user", username);
24         session.setMaxInactiveInterval(30 * 60);
25
26         resp.sendRedirect(req.getContextPath() + "/dashboard");
27     } else {
28         // لاگین ناموفق
29         req.setAttribute("error", "نام کاربری یا رمز عبور اشتباهه");
30         req.getRequestDispatcher("/login.jsp").forward(req, resp);
31     }
32 }
33 }

```

LoginFilter.java

```

1 java
2 @WebFilter(urlPatterns = {"/dashboard", "/admin/*"})
3 public class LoginFilter implements Filter {
4
5     @Override
6     public void doFilter(ServletRequest request, ServletResponse response,
7         FilterChain chain) throws IOException, ServletException {
8

```

14-5-Servlet

```
9  HttpServletRequest req = (HttpServletRequest) request;
10  HttpServletResponse resp = (HttpServletResponse) response;
11
12  HttpSession session = req.getSession(false);
13  boolean isLoggedIn = (session != null && session.getAttribute("user") != null);
14
15  if (isLoggedIn) {
16      chain.doFilter(request, response);
17  } else {
18      resp.sendRedirect(req.getContextPath() + "/login");
19  }
20 }
21 }
```

DashboardServlet.java

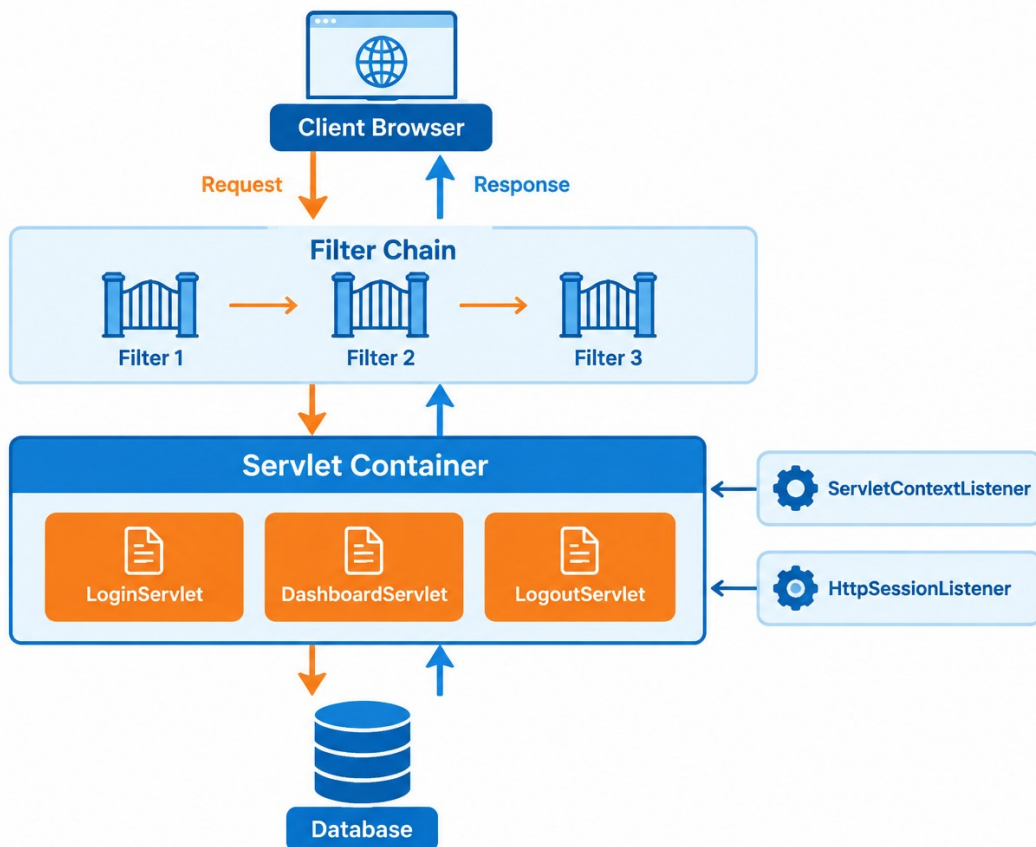
```
1 java
2 @WebServlet("/dashboard")
3 public class DashboardServlet extends HttpServlet {
4
5     @Override
6     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
7         throws ServletException, IOException {
8
9         HttpSession session = req.getSession(false);
10        String username = (String) session.getAttribute("user");
11
12        resp.setContentType("text/html");
13        resp.getWriter().println("<h1>خوش اومدی " + username + "</h1>");
14        resp.getWriter().println("<a href=" + req.getContextPath() + "/logout>خروج</a>");
15    }
```

}

LogoutServlet.java

```
1 java
2 @WebServlet("/logout")
3 public class LogoutServlet extends HttpServlet {
4
5     @Override
6     protected void doGet(HttpServletRequest req, HttpServletResponse resp)
7         throws IOException {
8
9         HttpSession session = req.getSession(false);
10        if (session != null) {
11            session.invalidate();
12        }
13
14        resp.sendRedirect(req.getContextPath() + "/login");
15    }
16 }
```

۱۴.۱۱ جمع‌بندی - نکات کلیدی مصاحبه



1. HTTP Stateless هست - برای نگه‌داشتن وضعیت از Session/Token استفاده می‌کنیم.
2. ServletContext برای کل اپ، ServletConfig برای به Servlet.
3. Filter قبل از Servlet اجرا میشه - برای احراز هویت، لاگ، فشرده‌سازی.
4. Listener به رویدادها گوش میده - برای راه‌اندازی منابع.
5. web.xml برای تنظیمات، Annotation برای مدرن بودن.
6. Session سمت سرور، Cookie سمت مرورگر.
7. JWT Stateless و مناسب REST API.
8. GET برای خواندن، POST برای تغییر.
9. curl ابزار خط فرمان برای تست HTTP.

10. load-on-startup باعث ساخت Servlet موقع استارت سرور میشه.

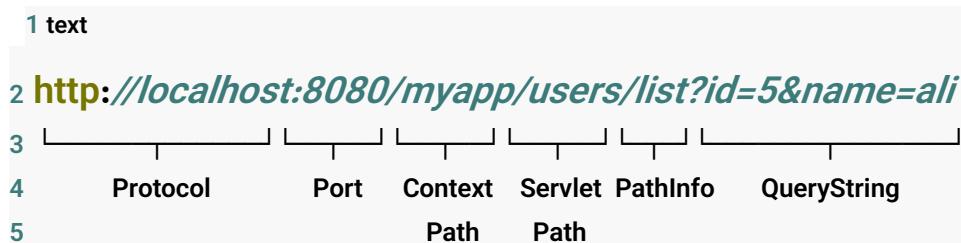
نکته طلایی: توی مصاحبه اگه پرسیدن "چرا از JWT استفاده می‌کنی؟" بگو: "چون Stateless هست و توی معماری Microservices و Cluster خیلی بهتر از Session عمل می‌کنه. Session توی Cluster نیاز به Sticky Session یا Session Replication داره که پیچیدگی رو زیاد می‌کنه."

getContextPath vs getServletPath vs getPathInfo vs getRequestURI

این چهار متد از پرکاربردترین‌های HttpServletRequest هستن و توی مصاحبه‌ها زیاد پرسیده میشن. اکثر برنامه‌نویس‌ها بینشون قاطی می‌کنن.

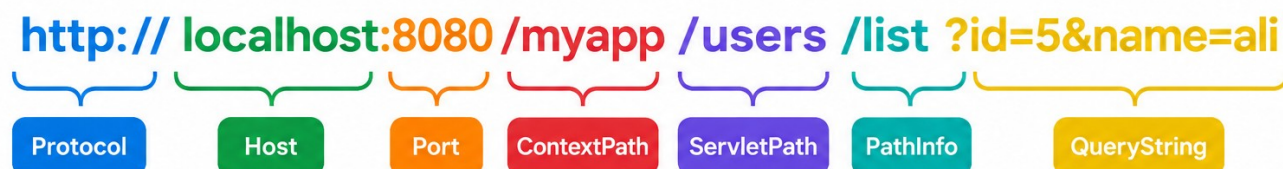
۱۴.۱۲ تفکیک URL توی Servlet

ساختار کامل یک URL



! سوال مصاحبه‌ای ۱۲: فرق `getContextPath`، `getServletPath`، `getPathInfo` و `getRequestURI` چیه؟

URL Anatomy



Java Method	Value
<code>getContextPath()</code>	<code>/myapp</code>
<code>getServletPath()</code>	<code>/users</code>
<code>getPathInfo()</code>	<code>/list</code>
<code>getRequestURI()</code>	<code>/myapp/users/list</code>

جواب:

متد	خروجی	توضیح
<code>()getContextPath</code>	<code>myapp/</code>	نام Context (نام WAR یا مسیر Deploy)
<code>()getServletPath</code>	<code>users/</code>	مسیری که به Servlet Map شده
<code>()getPathInfo</code>	<code>list/</code>	مسیر اضافه بعد از Servlet (اگه <code>*</code> باشه)
<code>()getRequestURI</code>	<code>myapp/users/list/</code>	کل URI (بدون Port، Protocol و QueryString)
<code>()getQueryString</code>	<code>id=5&name=ali</code>	پارامترهای Query
<code>()getRequestURL</code>	<code>http://localhost:8080/myapp/users/list</code>	کل URL (بدون QueryString)

مثال عملی کامل

فرض کن این تنظیمات رو داریم:

```
1 xml
2 <!-- web.xml -->
3 <servlet>
4   <servlet-name>userService</servlet-name>
5   <servlet-class>ir.nas.UserService</servlet-class>
6 </servlet>
7
8 <servlet-mapping>
9   <servlet-name>userService</servlet-name>
10  <url-pattern>/users/*</url-pattern>
11 </servlet-mapping>
```

و این درخواست:

```
1 text
2 http://localhost:8080/myapp/users/list?id=5&name=ali
3 توی UserService:
4 java
5 public class UserService extends HttpServlet {
6   @Override
7   protected void doGet(HttpServletRequest req, HttpServletResponse resp)
8     throws IOException {
9
10    System.out.println("ContextPath: " + req.getContextPath()); // /myapp
11    System.out.println("ServletPath: " + req.getServletPath()); // /users
12    System.out.println("PathInfo: " + req.getPathInfo()); // /list
13    System.out.println("RequestURI: " + req.getRequestURI()); // /myapp/users/list
14    System.out.println("RequestURL: " + req.getRequestURL()); // http://localhost:8080/myapp/users/list
15    System.out.println("QueryString: " + req.getQueryString()); // id=5&name=ali
16
```

```

17  //پارامترها
18  System.out.println("id: " + req.getParameter("id"));    // 5
19  System.out.println("name: " + req.getParameter("name")); // ali
20
21  //اطلاعات اضافی
22  System.out.println("Method: " + req.getMethod());      // GET
23  System.out.println("Protocol: " + req.getProtocol());   // HTTP/1.1
24  System.out.println("ServerName: " + req.getServerName()); // localhost
25  System.out.println("ServerPort: " + req.getServerPort()); // 8080
26  System.out.println("Scheme: " + req.getScheme());       // http
27  }
28 }

```

حالت‌های مختلف Mapping

حالت ۱: users/* (Path Mapping)/

```

1 text
2 URL: http://localhost:8080/myapp/users/list/5
3
4 getContextPath() = /myapp
5 getServletPath() = /users
6 getPathInfo() = /list/5 ← بقیه مسیر اینجا میاد
7 getRequestURI() = /myapp/users/list/5

```

حالت ۲: users (Exact Mapping)/

```

1 text
2 URL: http://localhost:8080/myapp/users
3
4 getContextPath() = /myapp
5 getServletPath() = /users
6 getPathInfo() = null ← چون مسیر اضافه‌ای نیست
7 getRequestURI() = /myapp/users

```

حالت ۳: *.do (Extension Mapping)

```

1 text
2 URL: http://localhost:8080/myapp/users.do
3
4 getContextPath() = /myapp
5 getServletPath() = /users.do
6 getPathInfo() = null
7 getRequestedURI() = /myapp/users.do

```

حالت ۴: / (Default Servlet)

```

1 text
2 URL: http://localhost:8080/myapp/users/list
3
4 getContextPath() = /myapp
5 getServletPath() = /users/list
6 getPathInfo() = null
7 getRequestedURI() = /myapp/users/list

```

کاربردهای عملی

۱. ساخت لینک نسبی

```

1 java
2 // ✓ استفاده کن getContextPath درست - همیشه از
3 String url = req.getContextPath() + "/users/list";
4
5 // ✗ عوض بشه، لینک میشکته Context غلط - اگه
6 String url = "/myapp/users/list";

```

۲. Redirect

```

1 java
2 // ✓ درست
3 resp.sendRedirect(req.getContextPath() + "/login");
4
5 // ✗ غلط
6 resp.sendRedirect("/login"); // اشتباه بره Context ممکنه به

```

۳. تشخیص مسیر توی Filter

```

1 java
2 public class AuthFilter implements Filter {
3     @Override
4     public void doFilter(ServletRequest request, ServletResponse response,
5         FilterChain chain) throws IOException, ServletException {
6
7         HttpServletRequest req = (HttpServletRequest) request;
8
9         String path = req.getRequestURI();
10        String contextPath = req.getContextPath();
11
12        // URI از ابتدای Context حذف
13        String relativePath = path.substring(contextPath.length());
14
15        System.out.println("Relative Path: " + relativePath);
16        // مثلاً: /users/list
17
18        // چک کردن مسیرهای عمومی
19        if (relativePath.startsWith("/public") ||
20            relativePath.startsWith("/css") ||
21            relativePath.startsWith("/js")) {

```

```

22     chain.doFilter(request, response);
23     return;
24 }
25
26 // چک کردن Login
27 HttpSession session = req.getSession(false);
28 if (session == null || session.getAttribute("user") == null) {
29     ((HttpServletResponse) response).sendRedirect(contextPath + "/login");
30     return;
31 }
32
33 chain.doFilter(request, response);
34 }
35 }

```

۴. استخراج Path Variable (قبل از Spring)

```

1 java
2 // URL: /myapp/users/123
3 // Mapping: /users/*
4
5 String pathInfo = req.getPathInfo(); // /123
6 if (pathInfo != null && pathInfo.length() > 1) {
7     String id = pathInfo.substring(1); // 123
8     Long userId = Long.parseLong(id);
9     // ...
10 }

```

! سوال مصاحبه‌ای ۱۳: چرا `getContextPath()` مهمه؟

جواب: چون اسم WAR یا Context Path می‌تونه توی محیط‌های مختلف فرق کنه:

```

1 text
2 Development: http://localhost:8080/myapp/...
3 Production: https://example.com/shop/...

```

اگه Hard-code کنی /myapp، توی Production میشکنه. ولی `getContextPath()` همیشه درست کار می‌کنه.

```
1 java
2 // درست ✓
3 String loginUrl = req.getContextPath() + "/login";
4
5 // غلط ✗
6 String loginUrl = "/myapp/login";
```

! سوال مصاحبه‌ای ۱۴: فرق `getRequestURI()` و `getRequestURL()` چیه؟

جواب:

متد	خروجی	شامل
<code>getRequestURI()</code>	myapp/users/list/	فقط مسیر (بدون Protocol/Port/Host/Query)
<code>getRequestURL()</code>	http://localhost:8080/myapp/users/list	مسیر کامل (بدون Query)

```
1 java
2 // URI: /myapp/users/list
3 // URL: http://localhost:8080/myapp/users/list
4
5 // ساخت Query کامل با URL
6 String fullUrl = req.getRequestURL().toString() +
7     (req.getQueryString() != null ? "?" + req.getQueryString() : "");
8 // http://localhost:8080/myapp/users/list?id=5&name=ali
```

! سوال مصاحبه‌ای ۱۵: getPathInfo() کی null برمی‌گردونه؟

جواب: وقتی:

1. Servlet با Exact Mapping تعریف شده باشه (users/)
2. Servlet با Extension Mapping تعریف شده باشه (do.*)
3. Servlet با Default Servlet (/) باشه
4. مسیر اضافه‌ای بعد از Servlet نباشه

فقط با Path Mapping (/users/*) مقدار داره.

```

1 java
2 // Mapping: /users/*
3 // URL: /myapp/users/list/5
4 getPathInfo() = /list/5 // ✓
5
6 // Mapping: /users
7 // URL: /myapp/users
8 getPathInfo() = null // ✗
9
10 // Mapping: *.do
11 // URL: /myapp/users.do
12 getPathInfo() = null // ✗

```

جدول خلاصه نهایی

متد	مثال	Context شامل	Servlet شامل	PathInfo شامل	Query شامل
()getContextPath	myapp/	✓	✗	✗	✗
()getServletPath	users/	✗	✓	✗	✗
()getPathInfo	list/	✗	✗	✓	✗
()getRequestURI	myapp/users/list/	✓	✓	✓	✗
()getRequestURL	http://localhost:8080/myapp/users/list	✓	✓	✓	✗
()getQueryString	id=5&name=ali	✗	✗	✗	✓

نکته طلایی مصاحبه

«همیشه برای ساخت URL از ()getContextPath استفاده کن، نه Hard-code. چون Context Path توی محیط‌های مختلف (Dev, Test, Prod) فرق می‌کنه و Hard-code باعث شکستن لینک‌ها میشه.»

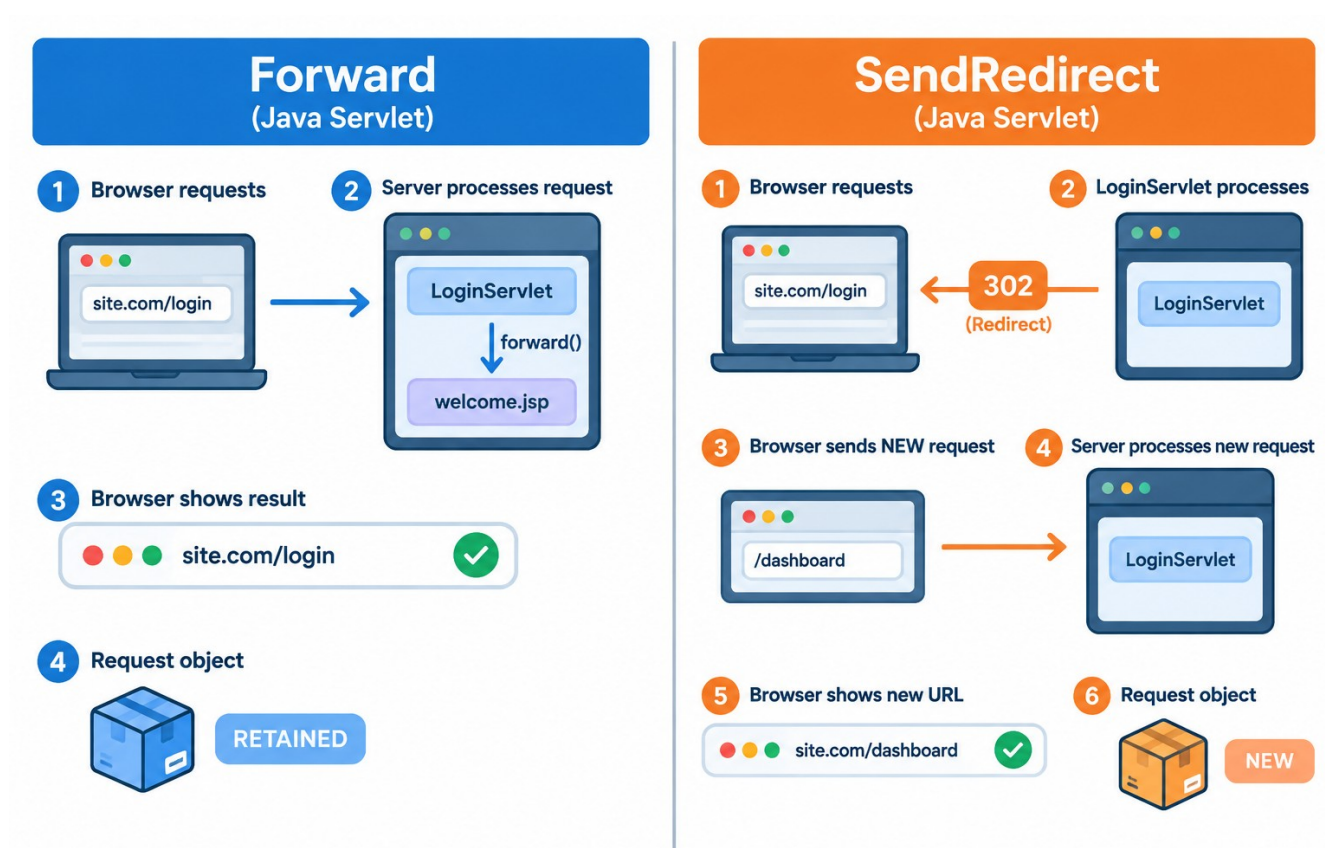
forward vs sendRedirect و getRequestDispatcher



این مفهوم یکی از پرتکرارترین سوالات مصاحبه‌های Servlet هست. خیلی‌ها بین forward و sendRedirect قاطی می‌کنن.

۱۴.۱۳ getRequestDispatcher چیه؟

()getRequestDispatcher یه راه برای پاس دادن درخواست به یه منبع دیگه (JSP یا Servlet دیگه) داخل خود سرور، بدون اینکه مرورگر بفهمه.



مثال واقعی

فرض کن کاربر می‌ره: site.com/login

قدم ۱: به Servlet به اسم LoginServlet این درخواست رو می‌گیره.

قدم ۲: می‌خوایم به جای اینکه خودمون HTML بسازیم، بریم به JSP که آماده داریم (welcome.jsp) رو نشون بدیم:

```

1 java
2 // LoginServlet.java
3 protected void doGet(HttpServletRequest req, HttpServletResponse res)
4     throws ServletException, IOException {
5
6     req.setAttribute("name", "علی"); // یه داده می‌فرستیم
7

```

```

8 req.getRequestDispatcher("/welcome.jsp") // JSP برو به این
9 .forward(req, res);           // کنترل رو بده بهش
10 }
1 jsp

2 <!-- welcome.jsp -->
3 <h1>سلام ${name}</h1>

```

چی اتفاق می‌افته؟

قبل	بعد
کاربر URL زده: site.com/login	✓ آدرس مرورگر همون site.com/login می‌مونه
	محتوای welcome.jsp نشون داده می‌شه
	name = "علی" قابل دسترسیه

خلاصه در یک خط

`getRequestDispatcher("/x").forward(req, res)` = «سرور، این درخواست رو ببر به x/ و جوابش رو بده به کاربر، ولی آدرس مرورگر رو عوض نکن.»

forward vs include

```

1 java
2 // forward: کنترل کامل می‌ره، برنمی‌گرده
3 req.getRequestDispatcher("/page.jsp").forward(req, res);
4
5 // include: محتوایش درج می‌شه، ولی ادامه می‌دی
6 req.getRequestDispatcher("/header.jsp").include(req, res);
7 res.getWriter().println("<p>متن اصلی</p>");

```

! سوال مصاحبه‌ای ۱۶: تفاوت forward و sendRedirect چیه؟ (خیلی مهم)

جواب:

```

1 java
2 // forward → حفظ می‌شه request، عوض نمی‌شه URL، داخل سرور
3 req.getRequestDispatcher("/welcome.jsp").forward(req, res);
4
5 // sendRedirect → جدید request، مرورگر می‌ره آدرس جدید
6 res.sendRedirect("welcome.jsp");

```

ویژگی	forward	sendRedirect
محل اجرا	داخل سرور	مرورگر (Client)
URL مرورگر	عوض نمی‌شه ❌	عوض می‌شه ✅
Request	همون request حفظ میشه ✅	request جدید ❌
Attribute ها	قابل دسترسی ✅	از دست میرن ❌
تعداد Request	۱	۲
سرعت	سریع‌تر ⚡	کندتر (رفت و برگشت به مرورگر)
کاربرد	پاس دادن به JSP/Servlet	رفتن به سایت دیگه یا بعد از POST

قانون طلایی

می‌خوای URL عوض نشه → forward
می‌خوای URL عوض بشه → sendRedirect

مثال کاربردی: Login

```

1 java
2 // ✅ عوض بشه URL کن Redirect، موفق Login درست - بعد از
3 if (isValidUser) {
4     HttpSession session = req.getSession(true);
5     session.setAttribute("user", username);
6     resp.sendRedirect(req.getContextPath() + "/dashboard"); // عوض میشه URL
7 }
8
9 // ✅ عوض نشه URL درست - نمایش فرم لاگین با خطا
10 if (!isValidUser) {
11     req.setAttribute("error", "نام کاربری اشتباه");
12     req.getRequestDispatcher("/login.jsp").forward(req, resp); // همونه URL
13 }

```

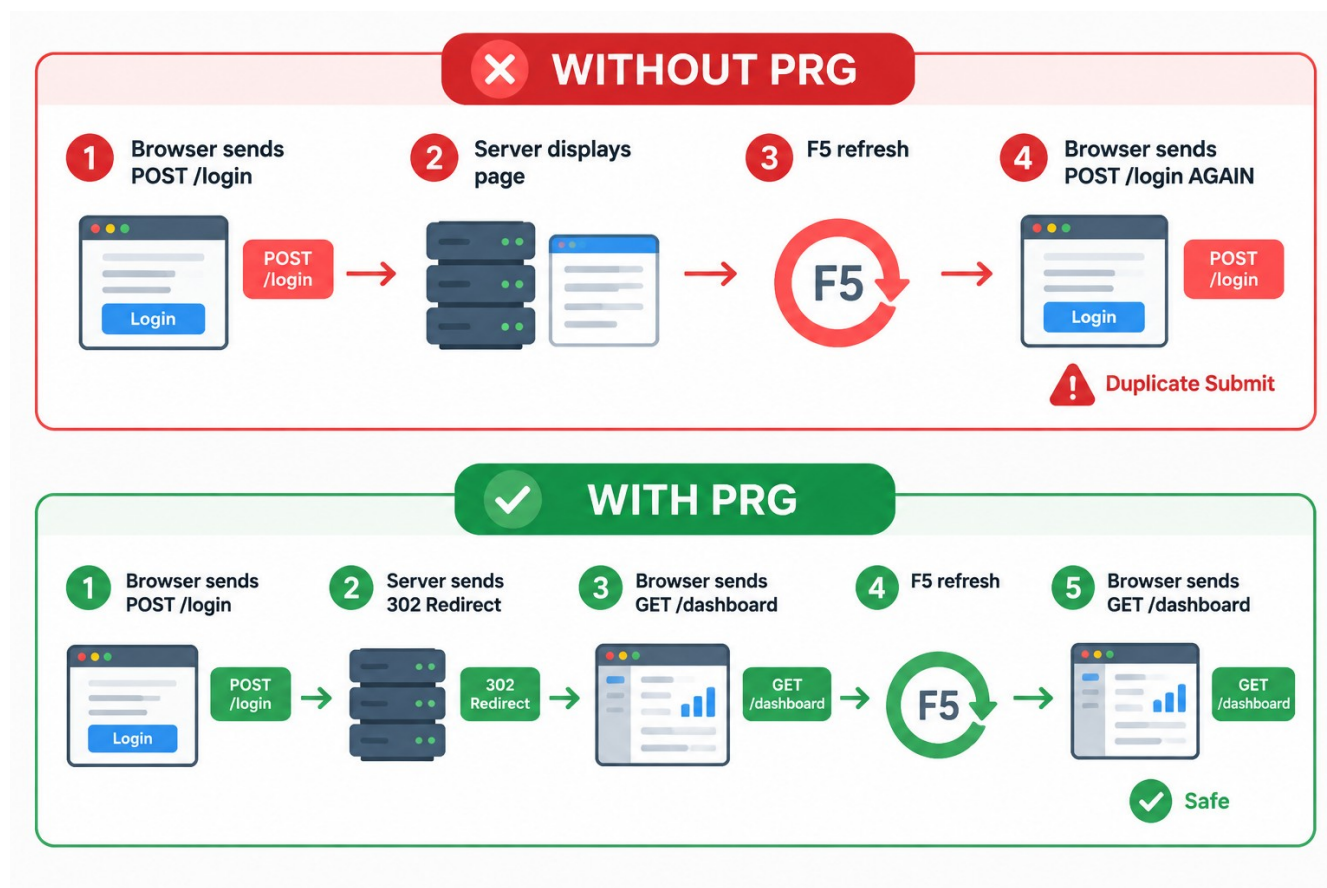
چرا بعد از Login موفق Redirect؟

• جلوگیری از Double Submit (کاربر F5 بزنه، دوباره Login نکنه)

• الگوی PRG (Post-Redirect-Get)

! سوال مصاحبه‌ای ۱۷: PRG Pattern چیست؟

جواب: Post-Redirect-Get الگو برای جلوگیری از ارسال دوباره فرم با Refresh هست.



1 text

2 ❌ بدون PRG:

3 POST /login → کاربر → نمایش صفحه F5 → POST دوباره Login)

4

5 ✅ با PRG:

6 POST /login → Redirect به /dashboard → GET /dashboard → کاربر F5 → GET فقط تکرار می‌شود

7 java

8 // پیاده‌سازی PRG

9 **protected void doPost**(HttpServletRequest req, HttpServletResponse resp)

10 **throws** IOException {

11 //۱. پردازش POST

12 processLogin(req);

13

14 //۲. Redirect (نه forward!)

15 resp.sendRedirect(req.getContextPath() + "/dashboard");

16 }

17

18 **protected void doGet**(HttpServletRequest req, HttpServletResponse resp)

19 **throws** ServletException, IOException {

20 //۳. نمایش صفحه

21 req.getRequestDispatcher("/dashboard.jsp").forward(req, resp);

22 }

! سوال مصاحبه‌ای ۱۸: چرا بعد از forward نمی‌تونی sendRedirect کنی؟

جواب: چون forward پاسخ رو Commit می‌کنه. اگه بعدش sendRedirect بزنی، IllegalStateException می‌گیری.

```
1 java
2 // ✗ خطا
3 req.getRequestDispatcher("/page.jsp").forward(req, res);
4 res.sendRedirect("/other"); // IllegalStateException!
5
6 // ✓ درست - یکیش رو انتخاب کن
7 if (condition) {
8     req.getRequestDispatcher("/page.jsp").forward(req, res);
9 } else {
10     res.sendRedirect("/other");
11 }
```

نکات تکمیلی

۱. getRequestDispatcher نسبی vs مطلق

```
1 java
2 // نسبی (به مسیر فعلی)
3 req.getRequestDispatcher("welcome.jsp");
4
5 // مطلق (از ریشه Context)
6 req.getRequestDispatcher("/welcome.jsp");
7
8 // ✓ همیشه مطلق استفاده کن (با / شروع کن)
```

۲. ServletContext از getRequestDispatcher

```

1 java
2 // (نسبی یا مطلق) Request از
3 req.getRequestDispatcher("/page.jsp");
4
5 // (همیشه مطلق) ServletContext از
6 getServletContext().getRequestDispatcher("/page.jsp");

```

۳. include برای Header و Footer

```

1 java
2 // layout.jsp
3 req.getRequestDispatcher("/header.jsp").include(req, res);
4 res.getWriter().println("<main>محتوا</main>");
5 req.getRequestDispatcher("/footer.jsp").include(req, res);

```

۴. forward بعد از getWriter()

```

1 1 java
2 2 // ❌ نمی‌تونی forward() getWriter() خطا - بعد از
3 3 res.getWriter().println("text");
4 4 req.getRequestDispatcher("/page.jsp").forward(req, res);
5
6 6 // ✅ بگیر writer کن، بعد forward اول
7 req.getRequestDispatcher("/page.jsp").forward(req, res);

```

جمع‌بندی

موضوع	forward	sendRedirect
محل	Server-side	Client-side
URL	ثابت	عوض میشه
Request	حفظ میشه	جدید
Attribute	حفظ میشه	از دست میره
سرعت	سریع	کند
PRG	✗	✓
Double Submit	✗ داره	✓ نداره